```
40.  # Initialize population
41.  population = []
42.  for _ in range(population_size):
43.      short_ma = np.random.randint(param_bounds['short_ma'][0], param_bounds['short_ma']
[1])
44.      long_ma = np.random.randint(param_bounds['long_ma'][0], param_bounds['long_ma'][1])
45.
46.      # Ensure short_ma is less than long_ma
47.      if short_ma >= long_ma:
48.          long_ma = short_ma + np.random.randint(10, 50)
49.
50.      population.append((short_ma, long_ma))
51.
52.  # Main genetic algorithm loop
53.  for generation in range(generations):
54.      print(f"Generation {generation+1}/{generations}")
55.
56.      # Evaluate fitness for current population
57.      fitness_scores = []
58.      for params in population:
59.          short_ma, long_ma = params
60.          fitness = evaluate_params(short_ma, long_ma)
61.          fitness_scores.append((params, fitness))
62.
63.      # Sort by fitness (higher is better)
64.      fitness_scores.sort(key=lambda x: x[1], reverse=True)
65.
66.      # Print best parameters in this generation
67.      best_params, best_fitness = fitness_scores[0]
68.      print(f"  Best parameters: short_ma={best_params[0]}, long_ma={best_params[1]}")
69.      print(f"  Best fitness (Sharpe): {best_fitness:.4f}")
70.
71.      # Select top performers for next generation
72.      top_performers = [params for params, _ in fitness_scores[:population_size//2]]
73.
74.      # Create next generation
75.      next_generation = []
76.
77.      # Keep top performers (elitism)
```